

UNIVERSITÀ DEGLI STUDI DI SALERNO

DIPARTIMENTO DI INFORMATICA



Tesi di laurea di I livello in
Informatica

Computer Music & Human-Computer

Interaction:

**gestione di messaggi MIDI, interfaccia grafica
e perfezionamento riconoscimento delle mani
in MarcoSmiles**

Relatori:

Roberto De Prisco

Rocco Zaccagnino

Candidato:

Salerno Daniele

Mat. 05121 05887

ANNO ACCADEMICO 2020/2021

Ai miei genitori, a mio fratello, a me stesso.

INDICE

1	INTRODUZIONE	4
2	RELATED WORKS	7
2.1	Theremin	7
2.2	KAiKU music glove	8
2.3	Mi.Mu Gloves	9
2.4	Genki Wave	9
2.5	AUMI	10
2.6	AeroMIDI	10
2.7	Virtual Piano	10
3	BACKGROUND	11
3.1	MIDI	11
3.1.1	Struttura e connessioni	12
3.1.2	Messaggi MIDI	13
3.2	Motore grafico Unity3D	15
3.3	Assets	16
3.4	Game Object	17
3.5	Scripting: la classe MonoBehaviour	17

INDICE

4	MARCOSMILES 3.0	20
4.1	Modulo per la gestione degli eventi MIDI	21
4.1.1	Messaggi Midi utilizzati in MarcoSmiles	21
4.1.2	Librerie C e software utilizzati	22
4.2	Progettazione e sviluppo	24
4.3	La classe MidiClass	25
4.4	Utilizzo del modulo in MarcoSmiles	31
4.5	Perfezionamento riconoscimento delle mani	34
4.5.1	Label errate nella fase di esecuzione	34
4.5.2	Scaling errato delle feature in fase di esecuzione	35
4.5.3	Reattività del sistema	40
4.6	Interfaccia grafica	40
4.6.1	Main Scene	40
4.6.2	Training Scene	41
4.7	Play Scene	43
5	CONCLUSIONI	46
5.0.1	Test effettuati	46
5.1	Lavori Futuri	49

CAPITOLO 1

INTRODUZIONE

In un ambiente in continua evoluzione sono molteplici gli aspetti di cui tener conto, tra cui la rivoluzione tecnologica a cui si assiste quotidianamente, l'attenzione verso un mondo del lavoro sempre più competitivo, una platea sempre più eterogenea di persone con esigenze sempre più diversificate. In un'era, dunque, in cui la tecnologia ha completamente rivoluzionato la vita quotidiana di ciascun individuo, garantire accesso ai dispositivi informatici di uso comune ad ogni tipo di utente è di fondamentale importanza. Usabilità e accessibilità sono requisiti rilevanti in fase di progettazione di un sistema informatico e sono oggetto di studio della Human Computer Interaction (HCI). Grazie a queste applicazioni le tecnologie si dimostrano non sostitutive di apparecchiature reali ma integrative degli elementi della realtà quotidiana: in questo caso, quindi, bisogna considerare il digitale come una ulteriore mediazione della realtà, cioè una riconfigurazione in un'altra chiave, il digitale non sostituisce nulla, ma contribuisce ad arricchire le nostre possibilità. Il costante sviluppo in questi ambiti ha portato al miglioramento di apparecchiature che permettono l'utilizzo di dispositivi elettronici a qualsiasi tipo di utenza, rendendo l'interazione più semplice, intuitiva e accattivante. Sono state oggetto di numerosi studi le Natural User Interfaces (NUI)[1],

il cui obiettivo è quello di superare le classiche interazioni fisiche legate all'utilizzo di mouse e tastiera, per sostituirle con altre più innovative, che prevedano l'impiego di azioni, gesti, e movimenti naturali per l'utente, al fine di controllare la “macchina” che sta utilizzando. La popolarità di tali interfacce, la diffusione di dispositivi di basso costo per il tracking dei movimenti, come il Leap Motion Controller[2] o il Kinect di Microsoft[3], e dispositivi che supportano il riconoscimento vocale come Amazon Echo Dot[4] o Google Nest[5] sono in costante aumento. Queste tecnologie hanno trovato impiego nei videogiochi, nella robotica, e l'home automation. MarcoSmiles è un sistema che adopera questo tipo di interfacce nell'ambito della Computer Music, un ramo della musica elettroacustica che impiega sistematicamente la programmazione su elaboratore numerico. Pietro Grossi, pioniere italiano dell'ingegneria, nel 1967, alle prime esperienze ufficiali di computer music presso la Olivetti General Electric a Pregnana Milanese dichiarò: “...*Ho preso il quinto Capriccio di Paganini, ho perforato le schede e, fatto il pacco di schede, consegnate al computer, il computer le suonava. Il computer ha suonato subito alla perfezione il testo che gli avevo dato, e immediatamente dopo, ho potuto fare di quei suoni quello che volevo. Questo per me era un salto incredibile. È stata un'emozione straordinaria...*” [6] [7]. Questo ambito è in continuo sviluppo e diversi musicisti hanno mostrato interesse verso tale tecnologia, in quanto offre la possibilità di creare musica in modo alternativo permettendo di personalizzare il modo di esprimersi “musicalmente”. MarcoSmiles utilizza una rete neurale artificiale e un dispositivo in grado di riconoscere il movimento delle mani e una rete neurale artificiale per permettere all'utente di suonare uno strumento musicale virtuale. Questa tesi, dunque, si pone il fine di illustrare le diverse integrazioni migliorative attuate, realizzate con l'intento di promuovere l'adozione di soluzioni efficaci ed accessibili con positive ricadute sui potenziali fruitori, e le nuove funzionalità aggiuntive introdotte:

- **Gestione di messaggi MIDI:** utilizzando lo standard MIDI è possibile far interfacciare MarcoSmiles con qualsiasi dispositivo dotato di interfaccia MIDI, trasformando quindi un semplice strumento musicale

1. INTRODUZIONE

virtuale in un vero e proprio controller MIDI in grado di convivere e scambiare informazioni in una Musical Network.

- **Interfaccia grafica:** il passaggio da un'interfaccia grafica bidimensionale ad una tridimensionale garantirebbe un'esperienza utente più immersiva; l'interfaccia grafica che si intende realizzare punta a simulare uno studio musicale, e se integrata con l'utilizzo di visori per la realtà virtuale potrebbe dare all'utente la sensazione di suonare realmente lo strumento musicale.
- **Perfezionamento riconoscimento mani:** una delle criticità di MarcoSmiles giace nell'impossibilità di effettuare una performance musicale, l'obiettivo è quindi quello di ottenere un prodotto funzionante da poter utilizzare per suonare delle melodie.

Nel capitolo 2 verranno descritti alcuni lavori distintivi del settore musicale, i quali si prefiggono di dare vita a strumenti che permettano di creare musica in maniera non convenzionale. Nel capitolo 3 verranno fornite nozioni di background. Nel capitolo 4, invece, sarà illustrata una descrizione del sistema che si intende realizzare, presentando gli obiettivi prefissati, e gli step effettuati durante la progettazione e lo sviluppo del sistema. Infine, nel capitolo 5 saranno esposti i risultati ottenuti e le possibili prospettive sui lavori futuri per estendere il sistema.

CAPITOLO 2

RELATED WORKS

Il presente capitolo illustra alcuni dispositivi che operano similamente al software MarcoSmiles e che quindi utilizzano il movimento delle mani per controllare il suono o per inviare messaggi MIDI. La principale differenza tra i sistemi illustrati di seguito e MarcoSmiles, riguarda la configurazione delle mani associata ad una determinata nota, la quale è scelta dall'utente in base alle sue preferenze e non preimpostata. L'utente dovrà quindi addestrare il sistema con le gestures che desidera adoperare, per poi successivamente utilizzarle per suonare in modo naturale, come se si trattasse di uno strumento reale.

2.1 Theremin

Il Theremin[8] è uno strumento musicale elettronico inventato nel 1919 dal fisico sovietico Lev Sergeevič Termen che non prevede il contatto fisico con l'esecutore. È composto da due antenne situate sopra e a lato di un contenitore nel quale sono alloggiare tutte le componenti elettroniche. Allontanando e avvicinando le mani alle antenne è possibile controllarne il suono: l'antenna superiore modula l'altezza, mentre quella laterale regola

2. RELATED WORKS

l'intensità. È considerato molto difficile da suonare in quanto non essendoci contatto fisico con esso, non esistono riferimenti visibili alla posizione relativa delle due mani.

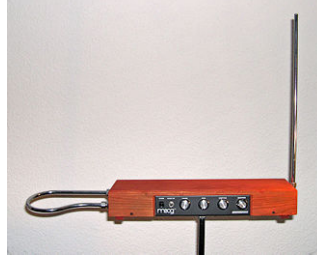


Figura 2.1: Theremin 8

Negli anni tale strumento è stato perfezionato e sono stati progettati diversi modelli dotati di interfaccia MIDI, come ad esempio il *MOOG Theremini* [9], in grado di interagire con la maggior parte degli strumenti musicali elettronici.

2.2 KAiKU music glove

il *KAiKU music glove*[10] è un dispositivo indossabile, il quale adopera sensori tattili, un giroscopio e un accelerometro, inoltre dispone di un software dedicato che permette di emulare diversi tipi di strumenti. È fondamentalmente un guanto su di cui sono disposti dei sensori che possono essere sfiorati con le dita o altri oggetti conduttori per inviare un segnale MIDI al dispositivo a cui è connesso, in modo che venga suonata la nota desiderata. Il musicista deve precedentemente assegnare ad ogni posizione sul guanto una determinata nota o un determinato messaggio MIDI.



Figura 2.2: KAiKU Glove (aggiungere sitografia)

2.3 Mi.Mu Gloves

I *Mi.Mu Gloves*[11] sono dei guanti che permettono di inviare messaggi MIDI in base al movimento delle mani. Hanno al loro interno una rete di sensori, in grado di percepire ogni singolo movimento come input digitale, il quale viene poi letto da un software e tradotto in output sonoro. Il guanto è completamente programmabile dal musicista a seconda dell'utilizzo che vuole farne. Quest'ultimo può associare ogni gesto ad un suono piuttosto che ad un altro.



Figura 2.3: Mi.Mu Gloves[11]

2.4 Genki Wave

Wave di Genki Instruments[12] è un anello indossabile che consente di controllare il suono, modellare effetti e inviare comandi ad una workstation audio digitale. Principalmente è possibile utilizzarlo per variare gli effetti applicati su di un suono (come ad esempio riverbero, distorsione e così via) effettuando un semplice movimento del dito o della mano.



Figura 2.4: Genki Wave[12]

2.5 AUMI

AUMI[13] è un software che permette all'utente di riprodurre suoni e frasi musicali attraverso movimenti e gesti, sfruttando la fotocamera del dispositivo sul quale è in esecuzione. Sebbene l'interfaccia AUMI possa essere utilizzata da chiunque, è stata concepita per bambini affetti da gravi disabilità fisiche. Il progetto tenta quindi di rendere la musica accessibile alla più ampia gamma possibile di individui.

2.6 AeroMIDI

AeroMIDI[14] è un controller MIDI che utilizza un Leap Motion Controller in grado di rilevare la posizione delle mani e che permette di creare un' interfaccia completamente personalizzabile, in cui è possibile mappare i messaggi MIDI a gesti delle mani, con la possibilità di controllarne fino a 6 per singola mano.

2.7 Virtual Piano

Virtual Piano[15] è un software basato sulla gesture recognition, il quale utilizza un Leap Motion Controller, e consente di suonare un pianoforte virtuale attraverso dei gesti delle mani predefiniti.

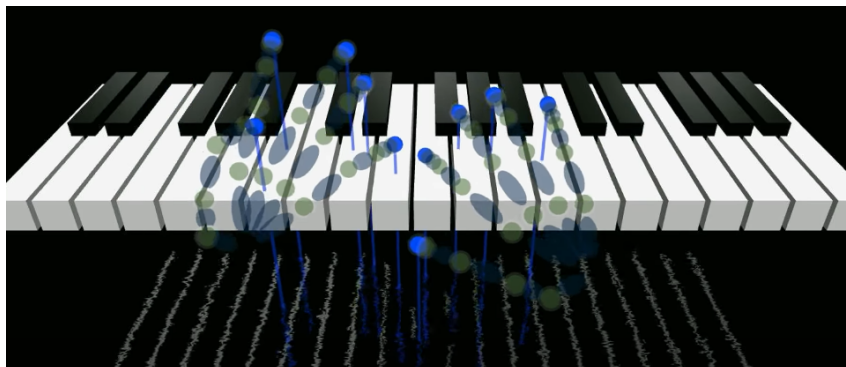


Figura 2.5: Virtual Piano[15]

CAPITOLO 3

BACKGROUND

3.1 MIDI

Il **MIDI**[16] (*Musical Instrument Digital Interface*) è un protocollo costituito da un insieme di messaggi e di regole per la comunicazione tra strumenti musicali elettronici e non solo. Nasce all'inizio degli anni ottanta e il primo prototipo fu presentato da due progettisti della Sequential Circuit (SCi), che proposero le prime specifiche MIDI attraverso il documento The Complete SCI MIDI. Si risolveva la problematica di far comunicare tra loro diversi strumenti musicali elettronici tenendo conto delle singole e specifiche caratteristiche. Esistevano già alcuni sistemi di interfacciamento ma garantivano il funzionamento solo su strumenti del medesimo costruttore in quanto basati su algoritmi proprietari. Al contrario il protocollo MIDI garantiva il funzionamento su dispositivi di costruttori diversi. Il protocollo nonostante negli anni successivi sia stato modificato e perfezionato nel tempo, a tutt'oggi la sua struttura si presenta quasi invariata, in quanto risulta sempre semplice da programmare, non richiede grandi risorse tecnologiche ed è ormai ampiamente diffuso in tutto il mondo.

3.1.1 Struttura e connessioni

Il protocollo MIDI è costituito da diversi messaggi ognuno dei quali è formato da parole (word) di uno o più byte che trasferiscono le azioni del musicista da uno strumento musicale (master) ad un altro (slave). I messaggi MIDI rappresentano tipicamente azioni relative alla pratica esecutiva, ad esempio: “*premi il pedale del Sustain*”, “*suona una nota della tastiera*” e così via. Descrivono quindi le azioni che un musicista può eseguire su uno strumento. Questi comandi vengono tradotti in messaggi digitali MIDI e viaggiano dal master allo slave dove vengono interpretati e associati ad un timbro. È importante notare che Il Messaggio MIDI non contiene alcun suono, non è un formato audio, ma riguarda solo le azioni del musicista. È costituito da valori discreti compresi tra 0 e +5 volt che sono quindi tradotti in sequenze di 0 e 1 organizzate in byte. Due dispositivi elettronici per comunicare tra loro tramite protocollo MIDI devono essere dotati di un’interfaccia adeguata, ovvero di hardware dotato di porte MIDI che consentano la trasmissione e la ricezione dei vari messaggi. L’interfaccia è composta da tre tipologie di connessione:

- **MIDI IN** che permette di ricevere i messaggi inviati da un’altra macchina;
- **MIDI OUT** che permette di inviare messaggi ad altri dispositivi;
- **MIDI THRU** che consente di reindirizzare un messaggio ricevuto verso un altro dispositivo.

Ad esempio, come illustrato nella Figura 3.1, in questo caso se il master invia un messaggio in uscita questo sarà ricevuto da entrambi gli slave.

3. BACKGROUND



Figura 3.1: Esempio connessioni MIDI[17]

3.1.2 Messaggi MIDI

Come già illustrato il messaggio MIDI è costituito da una parola (*word*) formata da diversi byte divisi in due tipologie:

- lo **status byte** che definisce il comando MIDI e quindi ne rappresenta il tipo di informazione (nota suonata, volume e così via) e indica il numero di canale su cui viaggerà;
- il **data byte** che rappresenta invece il valore richiesto per il parametro specificato nello status byte.

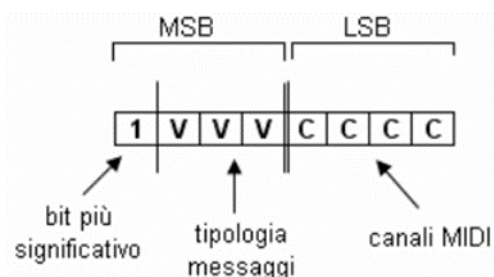


Figura 3.2: Struttura di uno *Status Byte*[17]

Il byte è suddiviso in due gruppi di 4 bit chiamati **MSB** (*Most Significant Bit*) e **LSB** (*Least Significant Bit*). Nel primo gruppo è presente il bit che indica la tipologia di Byte (1 se si tratta di uno Status Byte o 0 se si tratta di un Data Byte) mentre i tre rimanenti bit, indicati con la lettera V vengono utilizzati per definire i vari messaggi. Disponendo di 3 bit è possibile ottenere otto combinazioni possibili ($2^3 = 8$) illustrate nella figura 3.3. Gli ultimi 4 bit permettono di specificare su quale canale inviare il messaggio ($2^4=16$ canali).

3. BACKGROUND

Assegnando un canale ad uno strumento lo si identifica nel network musicale e permette quindi di inviare il messaggio al solo dispositivo interessato nel caso in cui ci siano diversi strumenti che possano avere accesso al messaggio. Ad esempio, nella Figura 3.1 se si vuole inviare un messaggio solo allo *slave 1* oppure solo allo *slave 2* basta assegnare a tali dispositivi canali diversi.

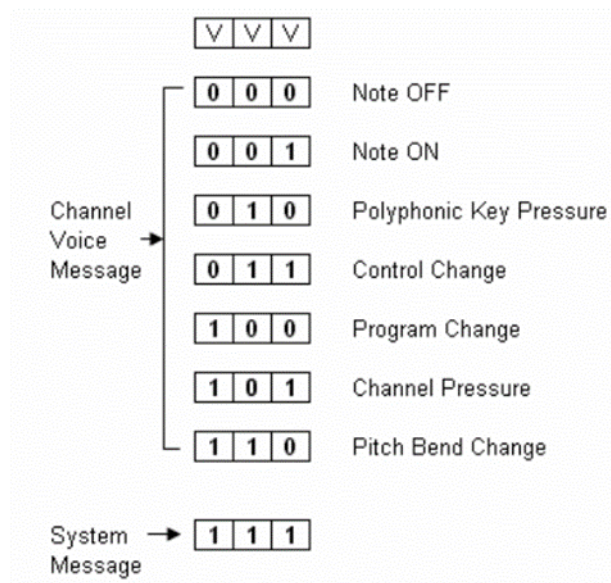


Figura 3.3: bit per la tipologia di messaggi

Si possono individuare due grandi categorie di messaggi MIDI:

- *Channel Message* (messaggi di Canale) i quali possono essere spediti su uno dei 16 canali MIDI
- *System Message* (messaggi di sistema) i quali sono indirizzati all'intero sistema di strumenti del network musicale

I **Channel Message** sono di due tipi:

- *Voice*: controllano le voci dello strumento (cosa deve suonare lo strumento)
- *Mode*: controllano come lo strumento deve gestire i Channel Voice Message

3. BACKGROUND

I **System Message** sono di tre tipi:

- *System Common*: riguardano tutti i ricevitori di un sistema indipendentemente dal canale
- *System Real Time*: usati per la sincronizzazione
- *System Exclusive*: sono messaggi “esclusivi” per ogni specifico strumento e dipendono dalla struttura di esso

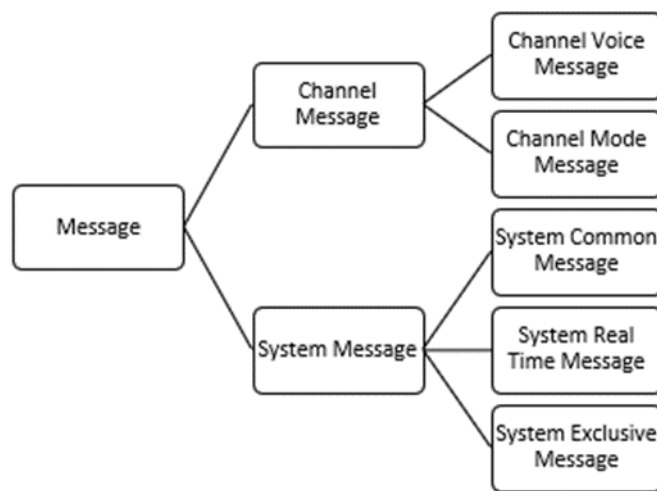


Figura 3.4: Messaggi MIDI

3.2 Motore grafico Unity3D

Unity3D è un motore grafico multiplatforma che consente lo sviluppo di videogiochi e altri contenuti applicativi, come visualizzazioni architettoniche e animazioni 3D in tempo reale. Permette di creare ambienti bidimensionali e tridimensionali. Offre, inoltre, la possibilità di sviluppare applicativi in grado di utilizzare la realtà aumentata o virtuale mettendo a disposizione diverse librerie per l'utilizzo di visori. L'Engine Unity3D semplifica la creazione di menu, l'esecuzione di animazioni, l'elaborazione di suoni e lo sviluppo di intelligenze artificiali. Inoltre, integra al proprio interno la possibilità di scrivere ed eseguire scripts in linguaggio C#. Offre un'interfaccia user friendly

completamente personalizzabile che permette di posizionare i diversi pannelli a proprio piacimento.

pannelli in Unity3D sono i seguenti:

- **Project:** permette di visualizzare e organizzare i diversi file all'interno del progetto, offrendo la possibilità di navigare fra le diverse cartelle.
- **Hierarchy:** permette di visualizzare la gerarchia degli elementi presenti in una determinata scena. É possibile trascinare gli elementi dal pannello "Project" al pannello "Hierarchy" per inserirlo nella scena corrente. Gli elementi che convivono nella scena sono chiamati GameObjects.
- **Inspector:** permette di ispezionare e modificare tutti gli attributi di un determinato GameObject, come la posizione nella scena, la rotazione o altre caratteristiche. Tali proprietà sono specificate attraverso i Components i quali vengono allegati ai GameObjects e permettono oltre che aggiungere proprietà, quali collisione con altri GameObjects, emissione di luce e così via, anche di incorporare degli script.
- **Console:** Permette di visualizzare la console di Unity3D.
- **Scene:** Permette di visualizzare in real time la scena, ovvero il "luogo" dove sono situati i diversi GameObjects attivi. É possibile quindi spostarsi all'interno di essa con il mouse o spostare, ruotare e ridimensionare i GameObject al suo interno.

3.3 Assets

Gli Assets rappresentano qualsiasi risorsa utilizzata dall'applicazione: modelli 3D, Audio, Textures, Fonts eccetera. Unity ha funzionalità built-in che consentono di modellare semplici oggetti come sfere o cubi, ma nel caso in cui si voglia lavorare con oggetti più complessi, l'ideale è importare modelli 3D all'interno del proprio progetto. L'Engine garantisce compatibilità con la maggior parte dei formati 3D, quali Maya, 3D Studio Max, Blender e

FilmBox, mantenendo tutti i materiali e le textures intatte. Per quanto riguarda i formati di immagini, supporta JPEG, PNG, TIFF oltre che file PSD. In merito ai file audio supporta formati quali WAV, MP3, OGG e AIFF. Importante sottolineare che Unity mette a disposizione un Asset Store ¹ dove è possibile acquistare o scaricare gratuitamente gli Assets da utilizzare nel proprio progetto.

3.4 Game Object

Ogni oggetto all'interno dell'applicazione è un *GameObject* e non è in grado di fare nulla al suo interno se non gli si conferiscono proprietà. Principalmente quindi è un contenitore in cui riporre funzionalità, le quali sono descritte dai *Components*. Un *GameObject* contiene sempre un *Component* di tipo *Transform* che non è rimovibile in quanto descrive la posizione e l'orientamento dell'oggetto all'interno della scena, lo sviluppatore decide poi, in base al tipo di oggetto che vuole ottenere, quali *Components* aggiungere. Ad esempio se si vuole realizzare un pulsante che alla pressione emette una luce rossa si può utilizzare il *Component* "Light", il quale permette di creare una luce, e in seguito, tramite script, azionarla. É possibile aggiungere o rimuovere *Components* o modificarne i parametri sia tramite il pannello "Inspector" sia tramite script utilizzando i metodi messi a disposizione dalla classe *GameObject* [19].

3.5 Scripting: la classe *MonoBehaviour*

La classe *MonoBehaviour*[20] offre un framework che consente di collegare gli script a *GameObjects* all'interno dell'editor, inoltre permette di intercettare specifici eventi attinenti al ciclo di vita degli oggetti semplificando lo sviluppo delle applicazioni. La classe *MonoBehaviour* è la classe base da

¹<https://assetstore.unity.com/>

3. BACKGROUND

cui deriva ogni script in Unity. Di seguito si elencano alcune delle funzioni più importanti della classe *MonoBehaviour*:

- ***Awake()***: viene chiamato quando avviene il caricamento dell'istanza dello Script e viene invocato solo una volta durante la durata di quest'ultimo. Principalmente viene utilizzato per inizializzare variabili precedentemente all'inizio dell'applicazione.
- ***Start()***: viene chiamato nel momento in cui uno script viene abilitato. Allo stesso modo di *Awake*, *Start* viene chiamato esattamente una volta nella durata dello script. Tuttavia, *Awake* viene invocato quando l'oggetto script viene inizializzato, a prescindere dal fatto che lo script sia abilitato o meno. *Start* invece necessita che lo script sia abilitato e quindi non può essere invocato sullo stesso frame di *Awake* a meno che lo script non sia abilitato al momento dell'inizializzazione. Persiste una relazione di precedenza tra le due funzioni: *Awake* viene su tutti gli oggetti nella scena prima che venga invocata la funzione *Start* di qualsiasi oggetto. Questa relazione risulta essere molto utile quando dati due oggetti X e Y, l'oggetto X necessita di alcune informazioni contenute nell'oggetto Y. In tal caso in *Awake* si dovrebbe inizializzare Y e in *Start* X.
- ***Update()***: viene chiamato ad ogni frame se lo script *MonoBehaviour* è abilitato ed è la funzione comunemente utilizzata per implementare funzionalità di scripting di qualsiasi tipo.
- ***FixedUpdate()***: simile alla funzione *Update*, ma svincolato dal Frame-Rate dell'applicazione. *Update* viene eseguito una volta per frame, mentre *Fixed Update* può essere eseguito una, zero o più volte per frame, a seconda delle preferenze impostate.

3. BACKGROUND

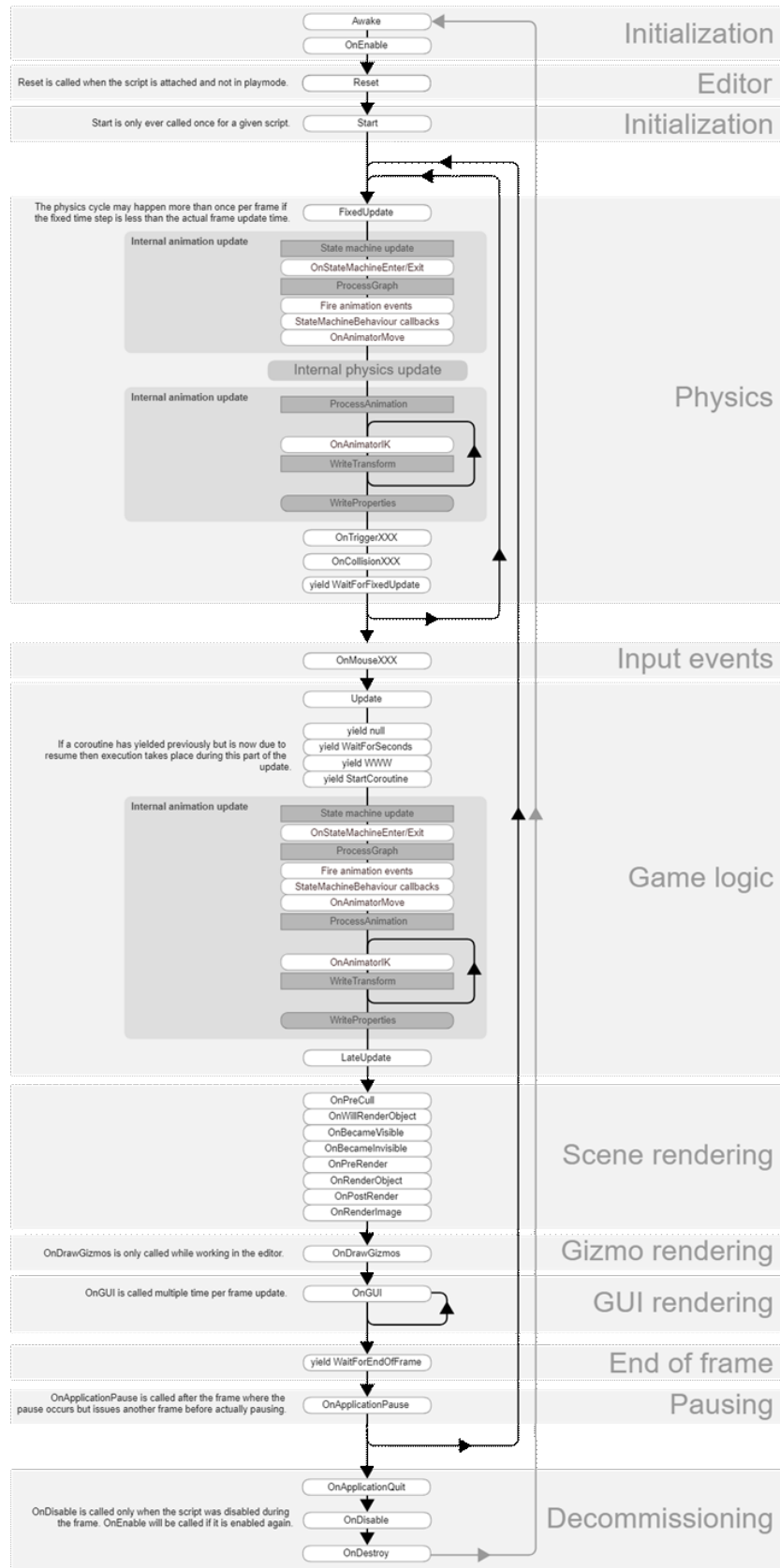


Figura 3.5: Lifecycle classe Monobehaviour[21]

CAPITOLO 4

MARCOSMILES 3.0

L'obiettivo principale di MarcoSmiles 3.0 è di ottenere un prodotto finale che garantisca maggiore usabilità e migliori performance rispetto alla versione precedente. MarcoSmiles 2.0 permetteva all'utente di generare del suono attraverso un sintetizzatore interno, il quale offriva la possibilità di variarlo mescolando diverse forme d'onda e aggiungendo ad esso diversi filtri. MarcoSmiles 3.0 si pone l'obiettivo di realizzare un vero e proprio controller MIDI, che possa essere collegato in una rete musicale e riesca a scambiare messaggi MIDI con altri dispositivi o poter essere utilizzato nelle Digital Audio Workstation come FLStudio, Cubase, Logic Pro X e così via. Le Digital Audio Workstation mettono a disposizione diversi plugin e sintetizzatori virtuali i quali accettano messaggi MIDI. L'utente, quindi, sarà in grado di accedere ad un vasto set di strumenti oltre che a sperimentare la produzione musicale utilizzando MarcoSmiles. Per quanto riguarda le modifiche relative all'interfaccia grafica l'obiettivo è quello di traslare da una bidimensionale ad una tridimensionale, con lo scopo di creare un Home Studio la cui esperienza utente potrà, in seguito, essere migliorata attraverso l'utilizzo di un visore per la realtà virtuale. Verranno effettuati tali miglioramenti facendo uso delle stesse tecnologie impiegate precedentemente: il motore grafico Unity3D e il

linguaggio C# per quanto riguarda l'interfaccia grafica e le funzioni di base, e la libreria SciKit Learn in Python per la parte di Machine Learning.

4.1 Modulo per la gestione degli eventi MIDI

4.1.1 Messaggi Midi utilizzati in MarcoSmiles

Nel capitolo 3 sono state illustrate le caratteristiche del protocollo MIDI e la struttura dei diversi messaggi disponibili. In questo paragrafo verranno descritti i principali messaggi MIDI utilizzati nello sviluppo del modulo incaricato della loro gestione illustrando anche il contenuto dei byte.

MIDI ON Rappresenta la pressione di un tasto sulla tastiera del master. Nel nostro caso non sono presenti tasti e tale evento viene inviato nel momento in cui si assume una configurazione delle mani relativa ad una specifica nota.

Status	1	0	0	1	C	C	C	C
1° Data	0	V	V	V	V	V	V	V
2° Data	0	V	V	V	V	V	V	V

Figura 4.1: Messaggio Note On

Vi sono due data byte:

- il primo rappresenta il numero della nota da suonare
- il secondo indica la velocity

MIDI OFF Rappresenta il rilascio di un tasto sulla tastiera del master. Nel nostro caso non sono presenti tasti e tale evento viene inviato nel momento in cui si cambia la configurazione delle mani e quindi si intende suonare una nota diversa da quella che si sta eseguendo oppure nel caso in cui si desideri rilasciare una nota che si sta suonando.

Status	1	0	0	0	C	C	C	C
1° Data	0	V	V	V	V	V	V	V
2° Data	0	V	V	V	V	V	V	V

Figura 4.2: Messaggio Note OFF

Vi sono due data byte:

- Il primo rappresenta il numero della nota da rilasciare
- Il secondo indica la velocity di rilascio

4.1.2 Librerie C e software utilizzati

Sanford.Multimedia.Midi

Sono diverse le librerie progettate appositamente per manipolare i messaggi MIDI. Nel nostro caso è stata scelta Sanford.Multimedia.Midi, un toolkit che permette di creare applicazioni MIDI in C#. Questa libreria consente di effettuare tutte le operazioni necessarie a un sistema MIDI come la creazione, l'invio e la ricezione di messaggi MIDI, oltre che la gestione dei dispositivi MIDI di input e di output. All'interno del toolkit sono presenti anche classi per la manipolazione di file MIDI nel caso in cui si voglia rendere permanente una performance musicale sottoforma di sequenze di messaggi MIDI. Per la realizzazione del modulo sopracitato sono state utilizzate le seguenti classi:

- *OutputDevice*: offre diversi metodi per gestire i dispositivi di output.
- *ChannelMessageBuilder*: permette di assemblare un messaggio MIDI facilitando la fase di definizione dei byte che lo compongono.

LoopMIDI

LoopMIDI è un software sviluppato da Tobias Erichsen [23] permette di creare porte MIDI virtuali per interconnettere applicazioni. Le porte create sono uniche per ogni utente ed esistono solo mentre l'applicazione loopMIDI è in esecuzione. Offre un'interfaccia molto semplice da utilizzare e consente di monitorare i flussi di byte che viaggiano su una determinata porta virtuale.

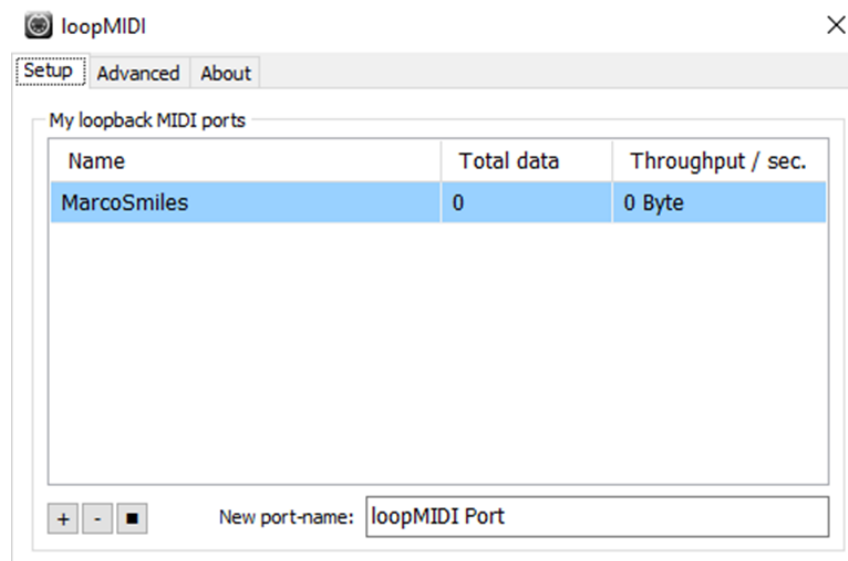


Figura 4.3: LoopMIDI

4.2 Progettazione e sviluppo

Prima di spiegare la fase di progettazione e sviluppo del modulo è necessario descrivere il tipo di prodotto finale che si vuole ottenere. Un controller MIDI (Alesis V49 ¹ in Figura 4.4) è un hardware in grado di utilizzare il protocollo MIDI, quindi, che genera e trasmette dati MIDI ad un dispositivo MIDI attivo.



Figura 4.4: Alesis V49

I due dispositivi per poter scambiare messaggi devono essere dotati quindi di un'interfaccia MIDI e di un canale fisico su di cui possano essere scambiati i messaggi.



Figura 4.5: Canale fisico MIDI

Analizziamo il funzionamento di una rete musicale composta da un controller MIDI e un pc su cui è in esecuzione una DAW (Digital Audio Workstation).

¹<https://www.alesis.com/products/view/v49>

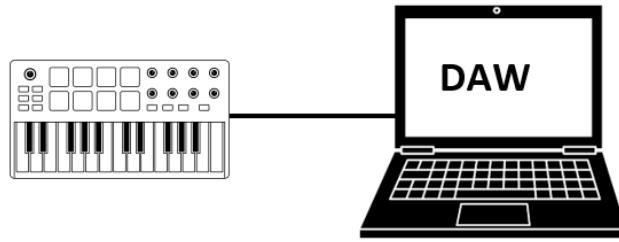


Figura 4.6: Esempio connessione MIDI

Il controller genera un messaggio MIDI e utilizza la propria porta MIDI OUT per inviarlo. Il pc riceve il messaggio sulla propria porta MIDI INPUT e lo elabora. MarcoSmiles essendo un software non dispone né di porte MIDI fisiche né di canali fisici, abbiamo quindi la necessità di realizzare delle porte virtuali. Per fare ciò è stato utilizzato il software LoopMidi spiegato nel paragrafo 4.1.2. La Digital Audio Workstation scelta per testare il corretto funzionamento del modulo è stata FL Studio[22].

4.3 La classe `MidiClass`

Il modulo per la gestione dei messaggi MIDI è stato inizialmente sviluppato in modo indipendente con il semplice obiettivo di inviare segnali di *NoteOn* e *NoteOff*. Data una lista di indici di note si prefiggeva di generare i messaggi MIDI appena citati e inviarli ad un dispositivo di Output. Il funzionamento del modulo è il seguente:

1. Ricerca della porta Output su cui inviare i messaggi
2. Costruzione e invio del messaggio

Per scambiare informazioni con una DAW è necessario creare non solo una porta virtuale di Output, ma anche una porta di Input. Il software LoopMIDI è stato impiegato per risolvere questo problema. Questo, infatti, crea 2 porte virtuali, una di Output e una di Input sulla quale viene replicato il messaggio inviato sulla porta di Output. Quindi è stata realizzata una connessione MIDI identica a quella illustrata nella 4.6, ma virtuale. Sono stati effettuati svariati

test per avere la certezza del corretto funzionamento del modulo che in seguito è stato integrato attraverso la classe MidiClass in MarcoSmiles la quale contiene tutti i metodi per la gestione dei Messaggi MIDI.

FindMidi - Esamina tutti i dispositivi MIDI output per trovare quello associato a MarcoSmiles.

```
1  public void FindMidi()
2  {
3      int DevId = 0;
4      //Ricavo il numero totale di Output Device
5      int numDevice = OutputDevice.DeviceCount;
6      for (int i = 0; i < numDevice; i++)
7      {
8          //Ricavo il nome della porta MIDI
9          //associata al dispositivo
10         dev = OutputDevice.GetDeviceCapabilities(i);
11
12         //trovo l'ID della porta
13         if (dev.name == "MarcoSmiles")
14         {
15             DevId = i;
16         }
17     }
18
19     //seleziono L'output Device e
20     //inizializzo Il MessageBuilder
21     outD = new OutputDevice(DevId);
22     builder = new ChannelMessageBuilder();
23
24 }
```

SendEvent - Riceve come parametri l'indice della nota da suonare e l'ottava di quest'ultima e invia il segnale di NoteOn relativo.

```
1 public void SendEvent(int note, int octave)
2 {
3     //Creazione del Messaggio MIDI da inviare
4     https://www.overleaf.com/project/61d6060a16ff475ba52dafd1
5     //Data1 rappresenta il primo Data byte
6     //in questo caso la nota da "suonare"
7
8     builder.Data1 = note + (octave * 12);
9
10    //Data2 rappresenta il secondo Data byte
11    //in questo caso la velocity
12    builder.Data2 = 105;
13
14    //Assegno il System Byte
15    //Most Significant Bit
16    //in questo caso il segnale di NoteOn (1001)
17    builder.Command = ChannelCommand.NoteOn;
18
19    //Least Significant Bit
20    //MidiChannel rappresente il canale Midi (0-15)
21    //a cui inviare il messaggio
22    builder.MidiChannel = 0;
23
24    //Quindi il System Byte inviato sar il seguente
25    // MSB | LSB
26    //1001 | 0000
27
28    //assemblo il messaggio
29    builder.Build();
30
31    //invio il messaggio al dispositivo di output
32    outD.Send(builder.Result);
33 }
```

Listing 4.1: SendEvent

SendMidiOff - Riceve come parametri l'indice della nota da rilasciare e l'ottava di quest'ultima e invia il segnale di NoteOff relativo.

```
1 public void SendMidiOff(int note, int octave)
2 {
3     //Creazione del Messaggio MIDI da inviare
4
5     //Data1 rappresenta il primo Data byte
6     //in questo caso la nota da "rilasciare"
7     builder.Data1 = (note + (octave * 12));
8
9     //Data2 rappresenta il secondo Data byte
10    //in questo caso la velocity di rilascio
11    builder.Data2 = 0;
12
13    //Assegno il System Byte
14    //Most Significant Bit
15    //in questo caso il segnale di NoteOff (1000)
16    builder.Command = ChannelCommand.NoteOff;
17
18    //Least Significant Bit
19    //MidiChannel rappresente il canale Midi (0-15)
20    //a cui inviare il messaggio
21    builder.MidiChannel = 0;
22
23    //Quindi il System Byte inviato sar il seguente
24    // MSB | LSB
25    //1000 | 0000
26
27    //assemblo il messaggio
28    builder.Build();
29
30    //invio il messaggio al dispositivo di output
31    outD.Send(builder.Result);
32 }
33 }
```

Listing 4.2: SendMidiOff

4. MARCOSMILES 3.0

Generazione degli indici delle note - Le note da suonare/rilasciare sono indicate nel primo Data byte che può assumere i seguenti valori:

Octave	Note numbers											
	Do	Do#	Re	Re#	Mi	Fa	Fa#	Sol	Sol#	La	La#	Si
	C	C#	D	D#	E	F	F#	G	G#	A	A#	B
0	0	1	2	3	4	5	6	7	8	9	10	11
1	12	13	14	15	16	17	18	19	20	21	22	23
2	24	25	26	27	28	29	30	31	32	33	34	35
3	36	37	38	39	40	41	42	43	44	45	46	47
4	48	49	50	51	52	53	54	55	56	57	58	59
5	60	61	62	63	64	65	66	67	68	69	70	71
6	72	73	74	75	76	77	78	79	80	81	82	83
7	84	85	86	87	88	89	90	91	92	93	94	95
8	96	97	98	99	100	101	102	103	104	105	106	107
9	108	109	110	111	112	113	114	115	116	117	118	119
10	120	121	122	123	124	125	126	127				

Figura 4.7: Indici note

MarcoSmiles è dotato di una Artificial Neural Network che, dopo essere stata addestrata dall'utente, è in grado di riconoscere una particolare configurazione delle mani alla quale è stata associata una nota. Fino alla versione 2.0 non era possibile “suonare” una pausa, tale funzionalità è stata implementata in MarcoSmiles 3.0. Dunque se precedentemente l'unico modo per rilasciare il suono era quello di rimuovere le mani dallo spettro visivo del Leap Motion Controller, ora è possibile addestrare il sistema a riconoscere una configurazione delle mani corrispondente ad una pausa. Nella versione 3.0, quindi, si ha la possibilità di addestrare il sistema su 2 ottave utilizzando 24 configurazioni delle mani (24 note) più una relativa alla pausa. Nella fase di Play è possibile selezionare il range di ottave in cui si desidera suonare. Avremo quindi un range di parametri che variano da 0 a 23 per gli indici delle note suonate e uno da 0 a 9 per gli indici delle ottave. Come illustrato nella Figura 10 il range delle note MIDI è compreso tra 0 e 127. Per ottenere quindi il valore relativo alla nota da inviare è stata utilizzata la seguente formula:

$$\text{notaMIDI} = \text{notaSuonata} + (\text{ottava} * 12)$$

Ipotizziamo di dover inviare un segnale di NoteOn per la nota F4 e analizziamone il processo di selezione dell'indice MIDI nei casi:

- *Ottava C3-C4:*
 - La rete Neurale restituirà il parametro **notaSuonata**=17
 - Il range di ottave selezionato ha indice 3
 - **notaMIDI**=17 + (3 *12) =53
- *Ottava C4-C5:*
 - La rete neurale restituirà il parametro **notaSuonata**=5
 - Il range di ottave selezionato ha indice 4
 - **notaMIDI**=5 + (4 *12) =53

Effettuando un riscontro sulla tabella dei valori associati ad ogni nota presente nella figura 4.7 è possibile verificare il corretto funzionamento della formula.

4.4 Utilizzo del modulo in MarcoSmiles

L'obiettivo è stato quello di integrare il modulo MIDI senza rimuovere il Synth Virtuale precedentemente sviluppato e utilizzabile in MarcoSmiles 2.0. Analizziamo brevemente il modo in cui è stato integrato il modulo e quali sono state le modifiche effettuate. Nella classe *_GM* viene acquisita la configurazione delle mani che l'utente ha appena assunto, ne viene calcolato l'indice relativo alla nota correlata e memorizzato nella variabile intera *indexPlayingNote* (contiene l'indice della nota da suonare).

```
1  if (isActive){
2      // Aggiorna l'array delle features currentFeatures in
3      //modo tale che venga calcolata la nota giusta
4      //ad ogni update
5      current_Features = TestingScript.GetCurrentFeatures();
6
7      // Salva in memoria l'indice dell'ultima nota suonata
8      indexPreviousNote = indexPlayingNote;
9
10     // Rappresenta la nota che deve essere suonata
11     indexPlayingNote = Mylabel[TestML.ReteNeurale(current_Features)];
12
13     //Cambia il colore del tasto sulla tastiera
14     //corrispondente alla nota che si sta suonando
15     ChangeColor(indexPreviousNote, indexPlayingNote);
16 }
```

Listing 4.3: *_GM* FixedUpdate

La classe *ProdeuralAudioOscillator* è incaricata di gestire la generazione sonora in MarcoSmiles 2.0 e si occupa di selezionare la frequenza da originare e di modulare, tramite effetti, il suono del Sinth. Nella nuova versione, grazie ad alcune modifiche apportate, sarà utilizzata anche per inviare messaggi MIDI. Nel metodo *Start()* oltre ad inizializzare le variabili relative alle diverse forme d'onda viene invocato il metodo *MidiClass.FindMidi()* illustrato precedentemente. Per il funzionamento del modulo MIDI sono state effettuate diverse variazioni al metodo *onAudioFilterRead* il quale, se precedentemente

si occupava di modificare le frequenze da far riprodurre al Synth, ora gestisce anche l'invio dei messaggi MIDI.

```
1 //Se l'utente ha attivato la funzione MIDI
2     if (MidiOn)
3     {
4         //se la nota da suonare risulta
5         //diversa da quella precedentemente suonata
6         if (OldNote != NoteToSend || OldOctave != octaveNumber)
7         {
8
9             //"rilascia" la nota precedentemente suonata
10            //inviando il messaggio MidiOff relativo
11            myMidi.SendMidiOff(OldNote, OldOctave);
12            //invia il messaggio MidiOn per la nota
13            //che si sta attualmente "suonando"
14            //controllo se l'indice inviato corrisponde
15            //effettivamente ad una nota
16            //o ad una pausa
17            if (NoteToSend != 24)
18            {
19                myMidi.SendEvent(NoteToSend, octaveNumber);
20            }
21            flagNote = 0;
22
23            //memorizzo la nota inviata
24            OldNote = NoteToSend;
25            OldOctave = octaveNumber;
26        }
27    }
28
29
```

Listing 4.4: Modifiche ProceduralAudioOscillator.onAudioFilterRead()

Dopo aver effettuato diversi test è stato riscontrato il seguente problema: il modulo non garantiva messaggi Midi rappresentanti note suonate per più tempo. MarcoSmiles effettua ripetutamente l'acquisizione della configurazione delle mani adottata dall'utente. Nel caso del Synth semplicemente si seleziona sempre la stessa frequenza creando un suono continuo. Nel caso dei messaggi MIDI, invece, è necessario inviare un solo messaggio MidiOn e nel momento in cui l'utente rimuove le mani dallo spettro visivo della Leap Motion o cambia configurazione delle mani inviare il MidiOff: il campo `flagNote` viene utilizzato per rilasciare l'ultima nota suonata nel primo caso, mentre nel secondo caso si effettua una verifica sulla nota che si sta "suonando" e quella che l'utente intende suonare (Listing 4.5).

```
1  //Se l'utente desidera non suonare alcuna nota
2      //invio il MidiOff relativo alla nota
3      //che si stava suonando precedentemente
4      if (flagNote == 0)
5      {
6          myMidi.SendMidiOff(OldNote, OldOctave);
7          flagNote = 1;
8      }
9
```

Listing 4.5: `ProceduralAudioOscillator.OnAudioFilterRead()`: Controllo nota da rilasciare

4.5 Perfezionamento riconoscimento delle mani

Di seguito verranno illustrati i problemi che caratterizzavano la versione precedente di MarcoSmiles e verranno descritte le soluzioni a tali problemi.

4.5.1 Label errate nella fase di esecuzione

Come accennato precedentemente, MarcoSmiles utilizza un Leap Motion Controller per rilevare la posizione delle mani, i quali dati rilevati vengono poi elaborati e interpretati da una rete neurale. Nella fase di addestramento l'utente associa ad una determinata nota una configurazione delle mani, e quindi ci si aspetta che nel momento in cui la si assuma nella fase di esecuzione venga riprodotta tale nota, ma questo nella versione 2.0 non si verificava. Prima di affrontare la soluzione a tale problema è necessario delineare il funzionamento del sistema, elencando tutti gli step che permettono il riconoscimento della posizione delle mani.

1. L'utente accede alla sezione "Train" e associa una configurazione delle mani alla nota F
2. Il sistema registra le features relative e le salva in un file csv
3. Avvia la fase di learning e attende la terminazione dello script python
4. Durante tale fase il sistema memorizza i seguenti file che saranno poi utilizzati per predire la nota in fase di esecuzione:
 - ***bias_out:*** contiene i valori relativi ai bias della rete neurale
 - ***min&max_values_dataset_out:*** contiene i valori minimi e massimi delle feature per effettuare in fase di esecuzione la normalizzazione MinMax
 - ***weights_out:*** contiene i valori relativi ai pesi della rete neurale

5. In fase di esecuzione, la classe TestML, ricostruisce la rete neurale in ambiente C# utilizzando i file appena elencati
6. Dopo aver ricostruito la rete neurale, in modo parametrico, riceve le feature relative alla posizione delle mani in real time e restituisce la nota predetta.

Il problema, dunque, giaceva nella mancanza di un file contenente le label utilizzate in fase di addestramento. Anche se la predizione era corretta non avendo a disposizione le label, il sistema si limitava a restituire un indice tra che variava tra il numero di note che l'utente ha addestrato. Nella nuova versione viene creato il file *lbl_notes* contenente le label necessarie per il corretto funzionamento del sistema. In fase di esecuzione quindi verrà effettuata un'associazione label-predizione in modo da restituire la nota esatta.

4.5.2 Scaling errato delle feature in fase di esecuzione

Dopo aver risolto la problematica sopracitata è stata riscontrata la seguente criticità: la configurazione assegnata in fase di training non era riconosciuta in modo corretto in fase di esecuzione. Ipotizziamo di aver assegnato alla nota F la seguente configurazione:

- Mano sinistra completamente aperta e con il palmo rivolto verso il basso
- Mano destra chiusa rivolta verso il basso

In fase di training abbiamo ottenuto il massimo dell'accuracy, quindi ci si aspetta un sistema in grado di riconoscere correttamente le diverse configurazioni ricevute in input. Al contrario, quando l'utente assumeva la configurazione descritta il sistema restituiva note casuali, anche nel caso in cui restava in posizione per più tempo. La soluzione è scaturita esaminando il modo in cui il sistema effettuava la fase di predizione, in particolare la classe TestML, la quale è tutt'ora incaricata di ricostruire la rete neurale in fase di esecuzione.

```
1 public static int ReteNeurale(float[] features)
2     //inizializzo un array di double dove memorizzo le feature scalate
3     double[] MyFeatures = new double[18];
4
5     MyFeatures = ScaleValues(features);
6
7     // output_hidden1 ha lo stesso numero di elementi di B1 BIAS LAYER1
8     var output_hidden1 = new double[B1.Count];
9     // output_hidden2 ha lo stesso numero di elementi di B2 BIAS LAYER2
10    var output_hidden2 = new double[B2.Count];
11
12    double x, w, r;
13
14    for (int i = 0; i < output_hidden1.Length; i++)
15    {
16        output_hidden1[i] += B1.ElementAt(i);
17
18        for (int j = 0; j < features.Length; j++)
19        {
20            x = MyFeatures[j];
21            w = W1[j][i];
22            r = x * w;
23
24            output_hidden1[i] += r;
25
26        }
27
28        if (output_hidden1[i] <= 0)
29            output_hidden1[i] = 0;
30
31    }
32
33    for (int i = 0; i < output_hidden2.Length; i++)
34    {
35        output_hidden2[i] += B2.ElementAt(i);
36
37        for (int j = 0; j < output_hidden1.Length; j++)
```

```
38         {
39             x = output_hidden1[j];
40             w = W2[j][i];
41             r = x * w;
42
43             output_hidden2[i] += r;
44
45         }
46     }
47
48     double sum = 0;
49     foreach (var item in output_hidden2)
50     {
51
52         sum += Mathf.Exp((float)item);
53     }
54
55     var toRet = new double[output_hidden2.Length];
56
57     for (int i = 0; i < output_hidden2.Length; i++)
58     {
59         toRet[i] = Mathf.Exp((float)output_hidden2[i]) / sum;
60     }
61
62     // index rappresenta la nota predetta
63     int index=toRet.ToList().IndexOf(toRet.Max());
64
65
```

Listing 4.6: TestML.ReteNeurale

Dunque dopo aver ottenuto le features in modo parametrico, viene effettuata una normalizzazione MinMax su di esse, ma i valori minimi e massimi adoperati in questa fase risultavano errati. Nello script Python viene creato il file `min&max_values_dataset_out` contenente i valori massimi e minimi di ciascuna features, ma nella versione precedente erano i seguenti:

```
1 scaler = MinMaxScaler()
2 train_scaled_D = scaler.fit_transform(training_set_data)
3 test_scaled_D = scaler.transform(test_set_data)
4 ycols = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17]
5 min_values = []
6 max_values = []
7
8 for col in ycols:
9     min_values.append(min(train_scaled_D[col]))
10    max_values.append(max(train_scaled_D[col]))
11
12 with open('min&max_values_dataset_out.txt', 'w') as f:
13     for item_min in min_values:
14         f.write(str(item_min) + " ")
15     f.write("\n")
16     for item_max in max_values:
17         f.write(str(item_max) + " ")
```

Listing 4.7: MarcoSmiles 2.0 salvataggio valori massimi e minimi

I valori memorizzati appartenevano a dataset su di cui era già avvenuta la normalizzazione MinMax e quindi nel momento in cui la classe TestML effettuava la normalizzazione otteneva feature non scalate correttamente. Nella nuova versione i valori vengono salvati precedentemente alla fase di scaling, adoperando il dataset originale. Dunque per prima cosa divide il dataset in modo stratificato in una sezione per il training ed una parte per il testing, e in seguito, prima di effettuare lo scaling, memorizza i valori necessari.

```
1 for col in ycols:
2     min_values.append(min(training_set_data[col]))
3     max_values.append(max(training_set_data[col]))
4 with open('min&max_values_dataset_out.txt', 'w') as f:
5     for item_min in min_values:
6         f.write(str(item_min) + " ")
7     f.write("\n")
8     for item_max in max_values:
9         f.write(str(item_max) + " ")
```

Listing 4.8: MarcoSmiles 3.0 salvataggio valori massimi e minimi

Il metodo incaricato di effettuare lo scaling delle features è ScaleValues, contenuto nella classe TestML, il quale riceve le features parametricamente e ne effettua lo scaling, restituendo un array di double contenente le features normalizzate.

```
1 private static double[] ScaleValues(float[] unscaledFeatures)
2 {
3     var scaledFeat = new double[unscaledFeatures.Length];
4     var minV = new double[unscaledFeatures.Length];
5     var maxV = new double[unscaledFeatures.Length];
6
7     for (int i = 0; i < unscaledFeatures.Length; i++)
8     {
9         //inizializzo i valori Minimi e Massimi relativi a ciascuna feature
10        minV[i] = double.Parse(min[i], CultureInfo.InvariantCulture);
11        maxV[i] = double.Parse(max[i], CultureInfo.InvariantCulture);
12    }
13
14    for (int i = 0; i < unscaledFeatures.Length; i++)
15    {
16        //normalizzazione MinMax
17        scaledFeat[i] = (unscaledFeatures[i] - minV[i]) / (maxV[i] - minV[i]);
18    }
19
20    return scaledFeatures;
21 }
```

Listing 4.9: TestML.ScaleValues

Utilizzando tali valori il sistema ora è in grado di riconoscere in modo accurato le configurazioni memorizzate dall'utente e predire correttamente la nota desiderata.

4.5.3 Reattività del sistema

Dopo aver trovato le rispettive soluzioni ai problemi descritti in precedenza, in fase di testing del sistema ne è stato riscontrato uno nuovo: MarcoSmiles 3.0 risultava estremamente sensibile al cambiamento della posizione delle mani; fondamentalmente nella fase di traslazione tra una configurazione e l'altra il sistema riproduceva delle note intermedie, in quanto il cambio di posizione delle mani, in questa fase, potrebbe far interpretare al sistema delle configurazioni relative ad una nota diversa e non desiderata. Per risolvere tale problematica è stato immesso un ritardo nella fase di scelta della nota da restituire, realizzato in modo tale da inviare la nota solo ed esclusivamente se la posizione relativa viene mantenuta per più tempo, in questo modo si evitano le note intermedie e si suona solo la nota che l'utente desidera eseguire.

4.6 Interfaccia grafica

L'interfaccia grafica è stata interamente sviluppata utilizzando il motore grafico Unity3D. Si è cercato di effettuare un restyling della precedente interfaccia bidimensionale, integrandola con nuovi elementi tridimensionali con l'obiettivo di ottenere un sistema piacevole da utilizzare e dal design più accattivante.

4.6.1 Main Scene

La Main Scene è l'home page del sistema. In alto è presente il logo di MarcoSmiles e di seguito i vari pulsanti che permettono di navigare le diverse schermate del sistema. Non sarà possibile accedere alla scena di esecuzione (Play Scene) senza aver precedentemente effettuato il training del sistema. In basso vi è una sezione che permette di accedere alle funzioni relative alle configurazioni del sistema che, attraverso dei pulsanti, consente di cambiare, importare ed esportare configurazioni precedentemente realizzate dall'utente.

Infine è presente, in alto a destra, il pulsante Quit per terminare l'esecuzione dell'applicazione.



Figura 4.8: Main Scene

4.6.2 Training Scene

La scena di training è dedicata all'addestramento della rete neurale. L'utente può associare ad ogni nota la configurazione delle mani che più desidera. La sezione centrale è dedicata alla proiezione delle mani in real time rilevate dal Leap Motion Controller. Sulla destra della schermata è presente un pulsante con l'icona della pausa di semiminima la cui funzionalità è quella di allenare il sistema a "suonare" una pausa. A sinistra invece è presente il pulsante info, il quale, una volta cliccato, mostra le istruzioni per l'addestramento del sistema. Nella parte bassa dell'interfaccia grafica è presente un pianoforte virtuale, composto da due ottave, il quale consente di selezionare la nota da addestrare. L'utente, tramite un click sul tasto del pianoforte, decide di addestrare il sistema su quella nota la quale assumerà un

colore blu, in modo da comunicare visivamente la scelta effettuata. Alla destra del piano sono presenti due pulsanti di colore differente:

- quello *verde* avvia la registrazione della configurazione delle mani
- quello *rosso* elimina l'ultima configurazione acquisita

A sinistra del pianoforte vi sono due pulsanti che permettono rispettivamente di tornare nella Main Scene (Figura 4.8), e di eliminare tutte le configurazioni delle mani registrate all'interno del sistema attuale. Infine centralmente nella parte superiore dell'interfaccia è presente un pulsante blu che ha la funzione di avviare lo script Python, il quale provvederà a finalizzare il processo di addestramento.

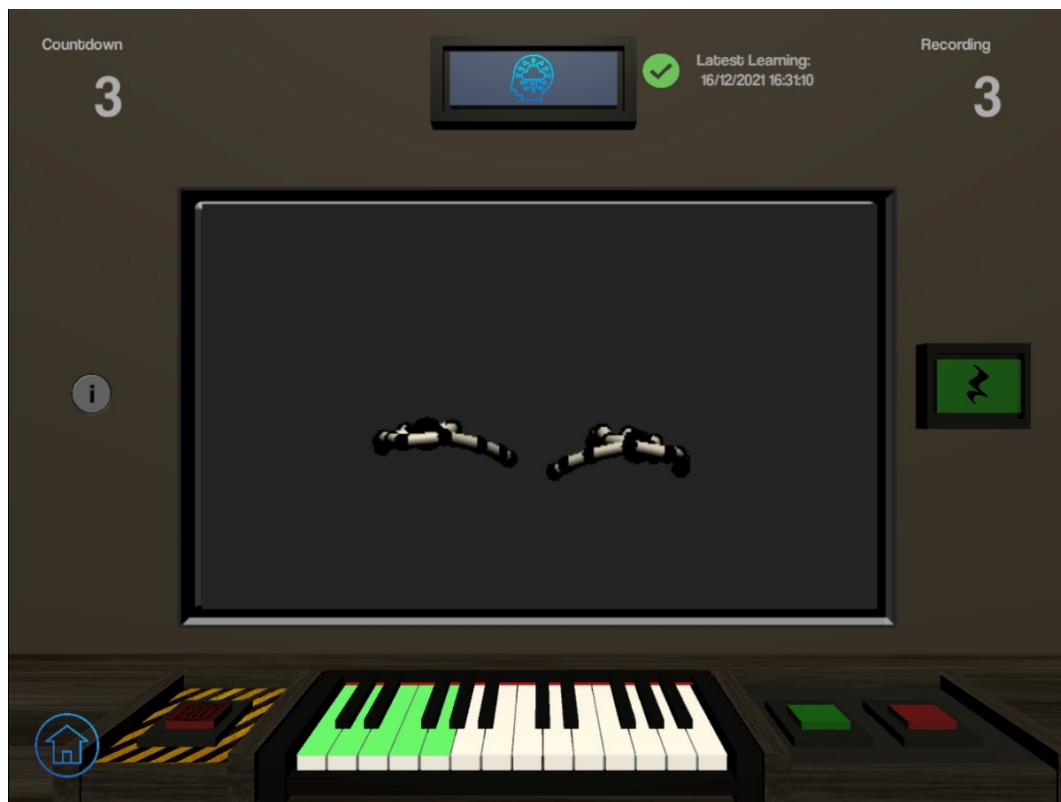


Figura 4.9: Training Scene

Il processo di addestramento si articola in più fasi:

1. l'utente seleziona, sul pianoforte virtuale, la nota che desidera addestrare (o la pausa con il relativo pulsante);

2. preme il tasto verde, il quale avvia un countdown di 3 secondi nel quale l'utente deve assumere la configurazione delle mani che desidera associare alla nota selezionata;
3. al termine del conto alla rovescia il sistema darà inizio alla effettiva registrazione della posizione delle mani; in questa fase verranno effettuate 500 misurazioni, per un totale di circa 3 minuti di addestramento (l'utente potrà visualizzare il progresso di tale operazione in alto a destra, dove è presente il relativo contatore);
4. terminato il processo la nota assumerà il colore verde ad indicare che per quest'ultima è stato effettuato l'addestramento

É possibile effettuare più addestramenti per una stessa nota. Nel caso in cui l'utente non risulti soddisfatto della configurazione appena registrata può eliminarla utilizzando il tasto rosso. Quando l'utente avrà registrato le varie posizioni desiderate, premerà il tasto blu, procedendo quindi con la creazione di una configurazione del sistema appena addestrato. Tale operazione porterà in esecuzione lo script python per il Machine Learning, il quale richiederà tempo proporzionale alle risorse computazionali della macchina sul quale è in esecuzione e alla dimensione del dataset. Al termine dell'esecuzione dello script il sistema consentirà all'utente di accedere alla scena di esecuzione (Play Scene) e di utilizzare, dunque, la configurazione appena creata. Infine, in alto, sarà indicato l'esito dell'apprendimento con il relativo orario di terminazione.

4.7 Play Scene

La Play Scene consente di eseguire performance musicali adoperando la configurazione del sistema selezionata. Come nella Training Scene (Figura 4.9) è possibile visualizzare in real-time le mani dell'utente. In basso è presente un pianoforte virtuale, i cui tasti vengono evidenziati in blu e ne viene simulata la pressione quando l'utente assume una configurazione delle mani che il sistema è in grado di riconoscere. Vi sono due pulsanti che permettono di variare il

range di ottave. L'opzione predefinita utilizza le ottave C3-C4 e i pulsanti permettono rispettivamente di aumentare o diminuire tale range. É presente anche un pulsante che permette di attivare/disattivare la funzionalità MIDI, la quale, una volta attivata, disattiverà l'audio del sintetizzatore e il pulsante emetterà una luce blu. Di fianco al pianoforte è presente un'ulteriore pulsante che permette di attivare/disattivare un "ologramma" il quale permette di visualizzare le note che si stanno eseguendo.



Figura 4.10: Play Scene

Accanto alla finestra in cui sono visualizzate le mani vi è la sezione dedicata a controllare i parametri del Synth Virtuale, il quale ha la caratteristica di essere totalmente personalizzabile dall'utente, e dunque consente di modificare il timbro del suono e di aggiungere modulazioni, filtri ed effetti. Grazie ad una interfaccia grafica basata sui Knobs, che tentano di simulare i potenziometri di un sintetizzatore reale, l'utente potrà variare i parametri del Synth.



Figura 4.11: Synth Virtuale

Sopra al pannello è presente un oscilloscopio che consente di visualizzare in real-time la forma d'onda generata dal sintetizzatore (*in Figura 4.10 risulta piatto in quanto la funzione MIDI è attiva e quindi il Synth non genera alcun segnale*). Infine in basso a sinistra vi è il pulsante per tornare al menu principale.

CAPITOLO 5

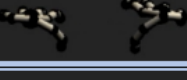
CONCLUSIONI

In questo capitolo conclusivo verranno illustrati gli obiettivi prefissati e quelli effettivamente raggiunti nel corso del lavoro svolto.

5.0.1 Test effettuati

Sono stati effettuati svariati test per verificare l'effettivo funzionamento del sistema. Il tempo di training indicato di seguito è relativo alla potenza computazionale della macchina che si sta utilizzando. In questo caso il processore utilizzato è il seguente: *Intel® Core™ i7-1165G7*[24]

5. CONCLUSIONI

Test	Note	Configurazioni		Righe Dataset	Tempo di Training	Training Accuracy	Comportamento del sistema
1	1	SOL		500	6m	100 %	Il sistema riconosce in modo preciso la configurazione assegnata ed assegna la label giusta in fase di esecuzione. Il messaggio Midi inviato alla DAW è corretto.
2	1 + pausa	DO		1000	15m	100 %	Il sistema distingue accuratamente le due configurazioni. Nessun problema rilevato con la funzionalità MIDI.
		PAUSA					
3	2+pausa	DO		1500	27m	100 %	Il sistema identifica le note assegnate alle configurazioni alla perfezione.
		RE					
		PAUSA					
4	5+pausa Configurazioni molto diverse tra loro	DO		2000	1h 27m	100 %	Il sistema riconosce le configurazioni in modo accurato. Si è provato ad eseguire una melodia ed è risultato semplice, senza occorrenza di errori.
		RE					
		MI					
		FA					
		SOL					
		PAUSA					
5	5+pausa Configurazioni simili (si è cercato di simulare l'esecuzione di un brano al pianoforte)	DO		2500	1h 35m	100 %	Il sistema identifica correttamente le prime 3 configurazioni, ovvero quelle dedicate al DO, al RE e al MI, ma presenta alcune difficoltà nelle ultime due. Come si può notare dalle immagini quest'ultime sono molto simili e il minimo movimento da parte dell'utente altera la predizione.
		RE					
		MI					
		FA					
		SOL					
		PAUSA					

Gestione di messaggi MIDI

Il lavoro è stato svolto con l'obiettivo di ottenere un prodotto finale in grado di interfacciarsi con dispositivi dotati di interfaccia MIDI, di realizzare un vero e proprio MIDI controller capace di inviare messaggi MIDI a tali dispositivi. Il modulo sviluppato, descritto nel paragrafo 4.1, garantisce tale funzionalità e risulta essere molto preciso nell'invio di tali messaggi. Essendo sviluppato con l'utilizzo della libreria C# `Sanford.Multimedia.Midi` non si sono verificati problemi di integrazione all'interno del sistema e del motore grafico Unity3D. Sono stati effettuati svariati test utilizzando la Digital Audio Workstation FL Studio i quali hanno sempre avuto esito positivo: i messaggi MIDI ricevuti dalla DAW corrispondevano alle note suonate dall'utente nel sistema. È stato quindi raggiunto l'obiettivo preposto.

Perfezionamento riconoscimento delle mani

Uno degli scopi del lavoro è stato quello di progettare un sistema molto più preciso di quello precedente in grado di funzionare correttamente e di ottenere, dunque, prestazioni migliori. Il problema fondamentale della versione 2.0 giaceva nell'impossibilità, per l'utente, di eseguire una performance musicale, anche poco complessa in quanto le configurazioni scelte dall'utente in fase di training non venivano individuate correttamente dal sistema. Dopo aver effettuato le modifiche descritte nel paragrafo 4.5 il sistema risulta essere più accurato nel riconoscimento della posizione delle mani e dopo aver addestrato MarcoSmiles sulla prima ottava, è risultato semplice eseguire alcune melodie senza l'occorrenza di errori da parte del sistema.

Interfaccia Grafica

L'intento è stato quello di creare un'interfaccia grafica in ambiente 3D cercando di simulare uno studio musicale. Utilizzando il motore grafico Unity3D è stata implementata tale interfaccia cercando di limitare, ove possibile, l'utilizzo di elementi bidimensionali. Tale interfaccia risulterebbe

più vantaggiosa se associata all'utilizzo di un visore per la realtà virtuale, in modo tale da proiettare l'utente all'interno dell'ambiente sviluppato.

5.1 Lavori Futuri

In questa sezione verranno esaminate alcune possibilità relative allo sviluppo di ulteriori funzionalità da implementare all'interno del sistema:

- **Tool di Realta Virtuale:** una possibile evoluzione del sistema come tool che sfrutta la realtà virtuale, in modo da creare un'esperienza utente ancora più coinvolgente. Il motore grafico Unity3D è utilizzato spesso in questo ambito, il che rappresenterebbe una facilitazione per tale miglioramento. Inoltre dotando il sistema di un visore per la realtà virtuale si riuscirebbe a sfruttare al meglio l'interfaccia grafica realizzata nella versione 3.0. Si potrebbe creare una visuale in prima persona dove l'utente, una volta indossato il visore, veda le proprie mani muoversi direttamente all'interno dello studio e non attraverso un display dedicato, e dunque possa effettuare specifici movimenti per modificare i parametri del Synth, per esempio ruotando i knobs, o premere i diversi comandi, simulando la pressione del tasto desiderato.
- **Impiego di più sensori:** è possibile immaginare uno scenario in cui convivano più sensori, di diverse tipologie, al fine di acquisire un maggior numero di informazioni relative alle gestures dell'utente, aumentando così la qualità delle predizioni della rete neurale.
- **Utilizzo di altre features:** grazie all'impiego di più sensori, si potrebbero aggiungere altri dati relativi configurazione delle mani, rendendo il sistema maggiormente personalizzabile, ampliando ulteriormente la varietà di a disposizione dell'utente.
- **Estensione del range di note suonabili:** si potrebbe estendere il range di note oltre le 2 ottave, dando vita ad un'esperienza ulteriormente simile ad uno strumento reale.

5. CONCLUSIONI

- **Suonare più note contemporaneamente:** attualmente il sistema consente di suonare una nota alla volta, si potrebbe, dunque, dare la possibilità all'utente di suonare più note contemporaneamente creando così degli accordi. Una soluzione potrebbe essere quella di associare ad una determinata configurazione delle mani uno specifico accordo.
- **Presets:** un altro possibile miglioramento può essere rappresentato dal garantire all'utente l'opportunità di salvare i vari parametri del synth, creando quindi dei preset utilizzabili in un secondo momento.
- **Personalizzazione interfaccia:** si potrebbe consentire all'utente di variare l'interfaccia, quindi muovere i diversi oggetti all'interno di essa o cambiarne alcuni, ad esempio potrebbe preferire un sassofono o una chitarra al posto del pianoforte digitale.
- **Sfruttare il Cloud computing per effettuare il training:** I calcolatori di fascia media non dispongono di elevata potenza di calcolo, impiegando quindi molto tempo per ultimare la fase di training del sistema; si potrebbero, dunque, sfruttare le grandi risorse computazionali e i servizi messi a disposizione dalle loro infrastrutture per minimizzare i tempi di addestramento.

BIBLIOGRAFIA

- [1] WIKIPEDIA - Natural user interface, https://en.wikipedia.org/wiki/Natural_user_interface
- [2] Leap Motion Controller, <https://www.ultraleap.com/product/leap-motion-controller/>
- [3] WIKIPEDIA - Microsoft Kinect, https://it.wikipedia.org/wiki/Microsoft_Kinect
- [4] WIKIPEDIA - Echo & Alexa , <https://www.amazon.it/b?ie=UTF8&node=15752736031>
- [5] Google Nest, https://store.google.com/it/product/nest_hub_2nd_gen?hl=it
- [6] WIKIPEDIA - Computer Music, https://it.wikipedia.org/wiki/Computer_music
- [7] Computer Music Pietro Grossi, http://www.hackerart.org/corsi/aba02/taddepera/computer_music.htm
- [8] WIKIPEDIA - Theremin, <https://it.wikipedia.org/wiki/Theremin>
- [9] Moog Music - Theremini, <https://www.moogmusic.com/products/etherwave-theremins/theremini>

BIBLIOGRAFIA

- [10] KAIKU Music Gloves, <https://www.kaikumusicglove.com/>
- [11] Mi.Mu Gloves, <https://mimugloves.com/>
- [12] Genki Wave, <https://genkiinstruments.com/products/wave>
- [13] AUMI, <http://aumiapp.com/about.php>
- [14] AeroMIDI, <https://aeromidi.net/index.php>
- [15] Virtual Piano, <https://gallery.leapmotion.com/virtual-piano-for-beginners/>
- [16] WIKIPEDIA Musical Instrument Digital Interface, https://it.wikipedia.org/wiki/Musical_Instrument_Digital_Interface
- [17] Hoepli Il protocollo MIDI, https://www.hoepli.it/editore/hoeppli_file/download_pub/978-88-203-3972-2_AppendiceA.pdf
- [18] Protocollo MIDI, <https://musicproduction.altervista.org/il-linguaggio-midi/>
- [19] Unity3D classe GameObject, <https://docs.unity3d.com/ScriptReference/GameObject.html>
- [20] Unity3D classe MonoBehaviour, <https://docs.unity3d.com/ScriptReference/MonoBehaviour.html>
- [21] MonoBehaviour lifecycle, <https://docs.unity3d.com/Manual/ExecutionOrder.html>
- [22] FL Studio, <https://www.image-line.com/>
- [23] LoopMIDI Tobias Erichsen, <https://www.tobias-erichsen.de/software/loopmidi.html>
- [24] Intel® Core™ i7-1165G7, <https://ark.intel.com/content/www/it/it/ark/products/208662/intel-core-i71165g7-processor-12m-cache-up-to-4-70-ghz.html>

BIBLIOGRAFIA

- [25] Human-Computer Interaction & Music: A Machine Learning approach for music performance -Daniele Perella
- [26] Human-Computer Interaction & Music: Sintetizzatore Sonoro In Unity - Gianluca Mannini
- [27] Human-Computer Interaction & Music: Unity3D and Leap Motion Controller - Antonio Manuele Gargiulo